Ledger Duplicate Verification — 7 October 2026

- **Date checked:** 2026-10-07
- **Database:** `ITMS-LV1`, table `ledger_entries`
- **Verified:** 2026-10-08
- **Reproduce with:** `node scripts/verify-ledger-duplicates.js 2026-10-07`

Conclusion

No duplicated ledger entries were posted on 7 October 2026.

All 663 entries are accounted for. Every test designed to catch a duplicate came back negative, including the tests that catch duplicates which do *not* look like repeated rows. One unrelated issue was found and is reported separately at the end.

What the day looks like

Measure	Value
Ledger rows	663
Distinct deal numbers	165
Total debits	471,687,169.09
Total credits	471,687,169.08
Deals accruing interest	159
Deals amortising	158

Row count against the surrounding fortnight — 7 October sits mid-range, with no volume spike of the kind a double posting would produce:

Date	Rows	Deals
Wed 24 Sep	640	162
Fri 25 Sep	652	159
Sat 26 Sep	604	153
Sun 27 Sep	604	153
Mon 28 Sep	670	163
Tue 29 Sep	628	158
Wed 30 Sep	706	172
Thu 01 Oct	672	166
Fri 02 Oct	648	163

Date	Rows	Deals
Sat 03 Oct	636	161
Sun 04 Oct	636	161
Mon 05 Oct	657	166
Tue 06 Oct	649	164
Wed 07 Oct	**663**	**165**

A duplicated EOD run would have put this near 1,300. It did not.

Why more than one test was needed

`ledger_entries` has **no unique constraint** — its only indexes are the primary key plus non-unique indexes on `transaction_id`, `account_id`, `deal_number` and `entry_date`. Nothing in the database physically prevents the same journal being inserted twice, so a duplicate had to be ruled out by inspection rather than assumed away by the schema.

A duplicate can also take three different shapes, and a single test only catches one of them:

1. an identical row inserted twice,
2. a re-post whose amount or account changed slightly (so the rows are not identical),
3. a **single** row carrying twice the correct amount, where there is no extra row at all.

Each shape is covered below.

Test results

1. Posting bursts — PASS

A second EOD run leaves a second large cluster of `created_at` timestamps against the same `entry_date`. There is only one such cluster:

created_at	Rows	ID range	What it is
2026-10-06 12:06	4	99209–99212	Repo maturity, booked a day ahead
2026-10-07 06:04	4	99215–99218	Buyback BB20261007001 leg 1 sale
2026-10-07 06:05	2	99219–99220	same journal, continued
2026-10-07 09:57	3	99221–99223	GSec purchase 20261007/GSEC/0003
2026-10-07 10:11	**650**	99224–99877	**the single EOD accrual/amortisation run**

One batch of 650, not two. The smaller clusters are deal-capture postings at the time each deal was approved, which is expected and is a different event type from anything the EOD batch wrote.

2. Exact duplicate rows — PASS (0 found)

Grouped every row by deal number, account, transaction ID, debit, credit **and** description. Zero groups returned more than one row.

3. Same deal + same event type + same side — PASS (0 true repeats)

This is the test that catches a re-post whose amount or account changed, which test 2 would miss. Descriptions were normalised to an event type (deal numbers and amounts blanked) so that a reworded repost still collides.

Six groups came back. **All six are single multi-leg journals, not repeats** — a journal that legitimately debits or credits more than one account. Each was checked by hand and balances:

Deal	Legs	Check
`20261006/REPO/0001`	DR 234,385,000.00 + 70,636.58	principal + interest; DR total = CR total ✓
`20261007/GSEC/0002` (BB-L1)	DR 18,269,472.20 + 0.20	= CR 17,665,638.55 + 603,833.85 ✓
`BB20260907001/BB-L2/SELL`	DR 2,188,435.80 + 93,077.73	= CR across cost, accrued, amortisation, gain ✓

The giveaway is that in a true duplicate every leg carries the **same** value. Here each leg carries a different value, because each represents a different component of one transaction. The script flags equal-value repeats automatically and found none.

4. One debit + one credit per accrual — PASS (0 of 317 wrong)

Daily accrual and daily amortisation must each post exactly one debit and one credit per deal per day. A double-post shows as two of either.

- Daily Accrual: **0 of 159 deals** posted anything other than 1 DR + 1 CR
- Amortisation: **0 of 158 deals** posted anything other than 1 DR + 1 CR

5. Day-over-day amount comparison — PASS (0 found)

This covers shape 3 above — a duplicate summed into one row, where no extra row exists to find. Every deal's 7 October accrual and amortisation was divided by its 6 October value.

- Deals at roughly 2× the previous day: **0**
- Deals moving more than 2% at all: **0** for both accrual and amortisation

Every amount moved smoothly day to day. Nothing doubled.

Two deals accrued on 7 October but not on 6 October: `20261007/GSEC/0002` and `20261007/GSEC/0003`. These are the two deals actually traded that day, so their first accrual falling on 7 October is correct.

6. Same security held under two deal numbers — PASS, reviewed by hand

A duplicated *holding* would accrue twice under two different deal numbers, which none of the tests above would see. Of 25 distinct ISINs accruing, three pairs matched the duplicate signature exactly — same ISIN, same face value, same value date, both `final_approved`, both accruing:

ISIN	Deals	Verdict
LKB00934F154	`20260415/GSEC/0013` + `/0017`	**genuine** — counterparties cc15 vs cc18, prices 98.3806 vs 97.7030, yields 11.05% vs 11.18%
LKB01533A154	`20260601/GSEC/0006` + `/0007`	**genuine** — same counterparty, but prices 93.8046 vs 93.3835, yields 12.60% vs 12.70%
LKB01533A154	`20260807/GSEC/0001` + `/0003`	**genuine** — counterparties cc15 vs cc26, prices 96.2495 vs 96.3353, yields 12.05% vs 12.03%

All three are real separate purchases of the same bond on the same day at different prices from different counterparties, booked minutes apart, each with its own remaining face value. Two positions in the same bond is normal trading, not a duplicate.

Separate finding: a 0.01 rounding imbalance

Not a duplicate, but it surfaced during the balance check and should be known.

On 7 October: deal `20261007/GSEC/0003` posts a purchase journal that is **0.01 out of balance**:

ID	Account	Debit	Credit	Description
99221	131-101-350-098-44	2,036,081.56		GSec Purchase — Treasury Bonds
99222	131-101-350-128-44	152,362.74		GSec Purchase — Accrued Interest
99223	131-101-410-164-44		2,188,444.29	GSec Purchase — Final Approval

Debits total 2,188,444.30 against a credit of 2,188,444.29. The settlement amount is rounded once, while the two components are rounded independently, so the halves can disagree by a cent.

This is long-standing and systemic, not new. Across the whole ledger, **90 of 25,973 journals** are out of balance, nearly all by exactly ±0.01, going back months. The buyback posting path already has a dedicated "(Rounding)" entry to absorb this — one appears on 7 October for `BB20260907001/BB-L2/SELL`. The plain GSec purchase path has no equivalent, which is why this cent is left stranded.

Five journals are out by more than 0.02 and are worth a separate look, since these are not rounding:

Date	Deal	Rows	Out by
2026-07-14	`20260601/GSEC/0005`	3	**13,812.15**
2026-06-06	`20241025/GSEC/0001`	3	**808.05**
2026-07-14	`20260612/GSEC/0002`	3	**346.75**
2026-06-16	`20260424/GSEC/0002`	3	4.87
2026-05-22	`20260522/GSEC/0004`	42	−0.04

Whole-ledger debits exceed credits by **14,971.86**, which is almost entirely those top three. None of this originates on 7 October.

Recommendations

1. **Nothing to correct for 7 October.** The day is clean. No entries should be reversed or removed.
2. **Add a unique constraint** on `ledger_entries` over `(deal_number, account_id, entry_date, description)` — or at minimum for the EOD accrual and amortisation paths. A duplicate is currently prevented only by application logic; the database would accept one. This is the same class of gap that allowed the duplicated mark-to-market deal found earlier.
3. **Give the GSec purchase path the rounding entry** the buyback path already has, so the remaining cent lands in a rounding account rather than leaving a journal unbalanced.
4. **Investigate the three material imbalances** (13,812.15 / 808.05 / 346.75). These are separate from the rounding pattern and each has only 3 rows, suggesting a leg failed to post.
5. **Run this check as part of EOD.** The script exits non-zero when it finds something, so it can gate the EOD job.

Evidence

Reproducible with:

```
node scripts/verify-ledger-duplicates.js 2026-10-07
```

Script: `scripts/verify-ledger-duplicates.js`. It is read-only — it runs `SELECT` statements only and writes nothing. It takes any date, so it can be pointed at any other day under question.

Final line of output for 7 October:

```
NO DUPLICATES FOUND.

Separately, 1 journal(s) are out of balance (see [6]).
```